ConveyorOS OAS — User & Technical Manual

Product: ConveyorOS Order Assembly System (OAS) **Version:** Current (April 2026) **Audience:** Operators, Managers, Administrators, IT Personnel

---

Table of Contents

1. Overview
2. System Requirements
3. First-Time Setup
4. User Guide — Non-Technical
   - 4.1 Logging In
   - 4.2 Dashboard
   - 4.3 Scanning Garments
   - 4.4 Completing an Order
   - 4.5 Printing Tickets
   - 4.6 Viewing Data (Customers, Orders, Garments)
   - 4.7 Clearing the Conveyor
   - 4.8 Logging Out
5. Manager / Admin Guide
   - 5.1 Creating Operators
   - 5.2 Operator Performance Data
   - 5.3 Slot & Occupancy Monitoring
6. Technical Reference — IT & Administrators
   - 6.1 Architecture Overview
   - 6.2 Technology Stack
   - 6.3 Database Schema
   - 6.4 Configuration & Settings
   - 6.5 CSV Integration (POS ↔ Conveyor)
   - 6.6 Hardware Integration
   - 6.7 Slot State Machine
   - 6.8 Session & Authentication Model
   - 6.9 Database Migrations
   - 6.10 Build & Deployment
   - 6.11 Log Files & Troubleshooting
7. Workflows — End-to-End
   - 7.1 Order Scanning Workflow
   - 7.2 CSV Order Import Workflow
   - 7.3 Conveyor Clearing Workflow
8. Roles & Permissions Reference
9. Glossary

---

1. Overview

ConveyorOS OAS (Order Assembly System) is a desktop/kiosk application designed for **dry cleaning and laundry facilities** that use a conveyor-based order fulfillment system. It bridges the gap between your Point-of-Sale (POS) system, your staff, and the physical conveyor hardware.

What it does:

- Receives customer orders automatically from your POS system via CSV file import.
- Allows operators to scan garment barcodes and assign them to numbered conveyor slots.
- Tracks every garment's location in real time across 100 conveyor slots.
- Communicates with the conveyor's Programmable Logic Controller (PLC) and OPC server to automate slot movement.
- Marks orders complete when all garments have been loaded, and triggers a printed receipt.
- Logs all activity by operator, enabling management to review performance.

Who uses it:

Role	Primary Use
Operator	Scan garments, process orders, clear the conveyor
Admin Operator	All operator tasks + manage staff accounts and settings
IT / Administrator	Install, configure, maintain the system and integrations

---

2. System Requirements

Workstation

Component	Minimum
OS	Windows 10 (64-bit)
CPU	Intel/AMD x64 or Apple Silicon
RAM	4 GB
Storage	500 MB free (application only)
Display	1280 × 800 or larger (touchscreen supported)

External Dependencies (must be installed separately)

Dependency	Notes
PostgreSQL 12 or later	The database server. Can be on the same machine or a network server.
OPC UA Server	Optional. Provides live conveyor status. System runs in degraded mode without it.
PLC / MODBUS device	Optional. Required for automated slot-jog commands.
ESC/POS Thermal Printer	Optional. Required to print order receipts automatically.
Barcode Scanner	USB HID or keyboard-wedge scanner recommended.

Network

- The workstation must reach the PostgreSQL server (default port `5432`).

- The workstation must reach the OPC UA server (default port `4840`) if OPC features are used.

---

3. First-Time Setup

The first time ConveyorOS OAS is launched with no prior configuration, a **Setup Wizard** will appear automatically. You must complete all three steps before the application will open normally.

> ***Note:*** *You will need your PostgreSQL connection details ready before starting the wizard.*

Step 1 — Database Connection

1. Enter your PostgreSQL **Host** (e.g., `localhost` or an IP address).
2. Enter the **Port** (default: `5432`).
3. Enter the **Database Name** — you must create this empty database in PostgreSQL beforehand.
4. Enter the **Username** and **Password**.
5. Click **Test Connection**. A green confirmation will appear if successful.
6. Click **Next** to continue.

> *On first successful connection, the application automatically creates all required tables and seeds 100 conveyor slots.*

Step 2 — POS CSV Input Directory

1. Click **Browse** and select the folder where your POS system writes order CSV files.
2. The application watches this folder every 5 seconds for new files.
3. Click **Next**.

Step 3 — Conveyor Output Directory

1. Click **Browse** and select the folder where conveyor activity CSV files should be written.
2. These files are consumed by downstream systems (e.g., reporting, sorter output).
3. Click **Finish**.

After completing the wizard, the app restarts into the **Login** screen and prompts you to create the first administrator account.

---

4. User Guide — Non-Technical

This section is written for **operators and managers** who use the system day-to-day. No programming or IT knowledge is required.

4.1 Logging In

1. When the app opens, you will see a keypad on the login screen.
2. Enter your **4-digit PIN** using the on-screen keypad or a physical keyboard.
3. Press the checkmark (confirm) button or press **Enter**.
4. If the PIN is correct, you are taken to the **Dashboard**.

> *If you have forgotten your PIN, ask an administrator to reset or recreate your account.*

4.2 Dashboard

The dashboard shows real-time system status at a glance.

Indicator	Meaning
OPC Connection	Green = conveyor system connected; Red = offline (limited functionality)
Hanger Sensor	Whether a garment is currently detected at the load point
Target Slot	The slot number the conveyor is currently targeting
Occupancy %	How full the conveyor is (0% = empty, 100% = all 100 slots occupied)
Items Today	Total garments scanned during the current day

If the OPC status shows **Offline**, you can still scan garments and manage orders, but automated slot-jog commands will not be sent to the conveyor hardware.

4.3 Scanning Garments

The **Garment Scanning** screen is where you will spend most of your time.

To scan a garment:

1. Navigate to the **Scan** screen from the sidebar.
2. Focus on the barcode input field (it is usually focused automatically).
3. Scan the garment's barcode with a barcode scanner, or type the barcode and press **Enter**.
4. The system responds with one of the following outcomes:

Outcome	What happened	What to do
Slot Assigned	The garment was placed on the conveyor and assigned a slot number.	Hang garment at the indicated slot.
Already on Conveyor	This garment was scanned before and is already assigned to a slot.	The system re-sends the jog command to move the conveyor to that slot.
Ticket Complete	This was the last garment on the order.	See Section 4.4.
Garment Not Found	The barcode does not match any order in the database.	Verify the barcode is correct; the POS order may not have imported yet.
Error	A database or hardware error occurred.	Note the error message and contact your administrator.

The screen also shows:

- **Customer Name** — the customer whose garment was just scanned.
- **Invoice Number** — the order number.
- **Item Description** — the type of garment (e.g., "Dress Shirt").
- **Progress** — how many of this order's garments have been loaded (e.g., "3 of 5").

4.4 Completing an Order

When you scan the **last garment** on a ticket, the system automatically:

1. Loads the final garment onto the conveyor.

2. Marks all garments on that order for **unloading**.
3. Changes the ticket status to **Complete**.
4. Sends a print command to the receipt printer.
5. Displays a **"Ticket Complete"** confirmation screen.

Acknowledge the completion by pressing the confirm button. The system returns to the scanning state, ready for the next garment.

> ***Important:*** *When a ticket completes, all garments belonging to that ticket will be signaled for conveyor unloading automatically. Ensure operators are ready to retrieve garments from the conveyor exit point.*

4.5 Printing Tickets

You can manually reprint a ticket at any time from the **Print Tickets** screen.

1. Navigate to **Print Tickets** from the sidebar.
2. Search by customer name, phone number, or invoice number.
3. Select the order from the results.
4. Click **Print**.

4.6 Viewing Data

The **Data** screen lets you browse all customers, orders, and garments.

Structure:

```
Customers
  └─ Orders (Tickets)
       └─ Garments
```

- Click a **Customer** to see all their orders.
- Click an **Order** to see all garments in that order and their current status.
- Garment status will show whether the item is still unprocessed, on the conveyor, or completed.

Use the search box at the top to filter by name or invoice number.

4.7 Clearing the Conveyor

Clearing removes all items from the conveyor and resets slot assignments. This is typically done at the end of a shift or when restarting operations.

> ***Caution:*** *Clearing cannot be undone. All garments must be physically removed from the conveyor before or during this operation.*

To clear the conveyor:

1. Navigate to the **Clear** option in the menu.
2. A confirmation dialog will appear.
3. Enter the confirmation code **123** and press confirm.
4. The system will send jog commands to route each occupied slot to the exit point.
5. Physically remove each garment as it arrives.
6. Once all slots are cleared, the system resets all slot assignments and ticket progress.

4.8 Logging Out

1. Click your name or the **Logout** button in the top bar.
2. Your session is closed and your garment/ticket counts are saved.
3. The app returns to the **Login** screen.

> *Always log out at the end of your shift so session data is recorded accurately.*

---

5. Manager / Admin Guide

Administrators have access to everything operators do, plus additional management tools.

5.1 Creating Operators

1. Navigate to **Settings** → **Operators**.
2. Click **Add Operator**.
3. Enter a **Username** and a **4-digit PIN** for the new operator.
4. Toggle **Admin** on if the operator should have administrator privileges.
5. Click **Save**.

> *The first operator account is created during initial setup and is always an administrator.*

5.2 Operator Performance Data

Navigate to **Operator Data** to view performance metrics.

Available timeframes: **Last 7 days**, **Last 30 days**, **Last 90 days**.

Metrics shown per operator:

- Total garments scanned
- Total tickets completed
- Average session duration

5.3 Slot & Occupancy Monitoring

The **Slot** view (accessible from the sidebar) shows the current state of all 100 conveyor slots.

Each slot is color-coded:

Color	State	Meaning
Gray	Empty	Slot is available
Yellow	Reserved	Slot is allocated to an upcoming order but not yet physically loaded
Green	Occupied	A garment is physically on this slot
Red	Blocked	Slot is temporarily unavailable
Orange	Error	Hardware or data inconsistency on this slot

You can click any slot to view which ticket and garment are assigned to it.

---

6. Technical Reference — IT & Administrators

This section is intended for **IT personnel, system integrators, and software engineers** responsible for installation, maintenance, and integration of ConveyorOS OAS.

6.1 Architecture Overview

ConveyorOS OAS is a **desktop application** built with the Tauri framework — a Rust backend bundled with a React/TypeScript frontend rendered by the operating system's native WebView.

```
┌──────────────────────────────────────────────────────────┐
│                  ConveyorOS OAS Process                  │
│                                                          │
│  ┌────────────────────┐      ┌──────────────────────────┐ │
│  │   React Frontend   │◄────►│      Rust Backend        │ │
│  │  (TypeScript + UI) │      │  (Tauri command handlers) │ │
│  └────────────────────┘      └──────────────────────────┘ │
│                                        │                  │
│                                        │                  │
│              ┌─────────────────────────┼────────────┐     │
│              │                         │            │     │
│   ┌──────────▼──┐  ┌──────────▼──┐  ┌─▼──────────┐  │     │
│   │ PostgreSQL │  │ OPC Client │  │ PLC (MODBUS) │  │     │
│   │ (Database) │  │  (OPC UA)  │  │  / Printer   │  │     │
│   └────────────┘  └────────────┘  └──────────────┘  │     │
│                                                     │     │
│              ┌──────────────────────────────────┐   │     │
│              │          File System Watcher     │   │     │
│              │   (CSV Import / CSV Export)      │   │     │
│              └──────────────────────────────────┘   │     │
└──────────────────────────────────────────────────────────┘
```

Communication pattern:

- The React frontend calls Rust functions via **Tauri IPC commands** (type-safe, async JSON messages over a local bridge).
- The Rust backend manages all database queries, hardware communication, and file I/O.
- Hardware status (OPC) is polled every 500 ms and pushed to the frontend.
- The CSV file watcher runs on a background thread, polling every 5 seconds.

6.2 Technology Stack

Frontend:

Technology	Version	Role
React	19.1	UI component framework
TypeScript	5.8	Type safety
Tailwind CSS	4.1	Utility-first styling
Vite	7	Build tool and dev server

Lucide React	—	Icon library

Backend (Rust):

Crate	Version	Role
tauri	2	Desktop app framework
diesel	2.2	PostgreSQL ORM
tokio	1	Async runtime
tokio-modbus	—	MODBUS protocol client
open62541	0.10	OPC UA client
melsec_mc	0.4.13	Mitsubishi PLC protocol
escpos-rs	0.4.3	ESC/POS thermal printer
serde	—	JSON serialization
chrono	0.4	Date/time handling

Storage:

Technology	Role
PostgreSQL 12+	Primary operational database
Tauri Plugin Store	Settings persistence (JSON file)

6.3 Database Schema

The system uses **8 tables**. Migrations are applied automatically on startup.

`users`

Stores operator accounts.

Column	Type	Notes
`id`	SERIAL PK	Auto-increment
`username`	TEXT	Operator display name
`pin`	TEXT	4-digit PIN (stored as text)
`is_admin`	INT	1 = administrator, 0 = standard
`created_at`	TIMESTAMP	Account creation time

`customers`

Stores customer records imported from POS.

Column	Type	Notes
`id`	SERIAL PK	—
`customer_id`	TEXT	External POS customer identifier
`first_name`	TEXT	—
`last_name`	TEXT	—
`phone_number`	TEXT	—
`created_at`	TIMESTAMP	—

`tickets`

Represents a customer order (invoice).

Column	Type	Notes
`id`	SERIAL PK	—
`full_invoice_number`	TEXT	Full invoice ID from POS
`display_invoice_number`	TEXT	Human-readable version
`number_of_items`	INT	Total garments in this order
`customer_identifier`	TEXT	FK → `customers.customer_id`
`customer_first_name`	TEXT	Denormalized for display
`customer_last_name`	TEXT	Denormalized for display
`customer_phone_number`	TEXT	Denormalized for display
`invoice_dropoff_date`	TIMESTAMP	When customer dropped off
`invoice_pickup_date`	TIMESTAMP	When customer expects pickup
`garments_processed`	INT	Garments loaded so far
`ticket_status`	TEXT	`Not Processed` / `Processing` / `Complete`
`created_at`	TIMESTAMP	—

`garments`

Individual garment items within a ticket.

Column	Type	Notes
`id`	SERIAL PK	—
`full_invoice_number`	TEXT	FK → `tickets.full_invoice_number`
`display_invoice_number`	TEXT	—

`item_id`	TEXT	Barcode / unique garment ID
`item_description`	TEXT	e.g., "Dress Shirt", "Pants"
`invoice_dropoff_date`	TIMESTAMP	—
`invoice_pickup_date`	TIMESTAMP	—
`invoice_comments`	TEXT	Special handling notes
`slot_number`	INT	Current slot (-1 = unloaded/complete)

`slots`

The 100 physical conveyor slots.

Column	Type	Notes
`slot_number`	INT PK	1 through 100
`slot_state`	TEXT	`empty` / `reserved` / `occupied` / `blocked` / `error`
`assigned_ticket`	TEXT	Which ticket is using this slot
`item_id`	TEXT	Which garment is on this slot
`created_at`	TIMESTAMP	—
`updated_at`	TIMESTAMP	Last state change

`sessions`

Tracks each operator login session.

Column	Type	Notes
`id`	SERIAL PK	—
`user_id`	INT	FK → `users.id`
`login_at`	TIMESTAMP	Session start
`logout_at`	TIMESTAMP	Session end (NULL = still active)
`garments_scanned`	INT	Count during this session
`tickets_completed`	INT	Count during this session
`is_admin`	INT	Snapshot of role at session time

`conveyoractivity`

Immutable audit log of every slot operation.

Column	Type	Notes
`id`	SERIAL PK	—

`user_id`	INT	FK → `users.id`
`item_id`	TEXT	Garment barcode
`full_invoice_number`	TEXT	Order reference
`slot_number`	INT	Slot affected
`time_stamp`	TIMESTAMP	When the event occurred

`app_state`

Single-row application state table.

Column	Type	Notes
`id`	INT PK	Always 1
`last_used_slot`	INT	Last slot allocated
`num_items_on_conveyor`	INT	Current occupancy count
`updated_at`	TIMESTAMP	—

6.4 Configuration & Settings

Settings are persisted using `tauri-plugin-store` in a JSON file at:

Platform	Path
Windows	`%APPDATA%\com.whiteconveyors.conveyoros-oas\`

Settings fields:

Key	Type	Description
`dbHost`	string	PostgreSQL host
`dbPort`	string	PostgreSQL port (default `5432`)
`dbName`	string	Database name
`dbUser`	string	Database username
`dbPassword`	string	Database password
`posCsvDir`	string	Absolute path to the POS CSV import directory
`conveyorOutputDir`	string	Absolute path for conveyor CSV output
`opcEndpointUrl`	string	OPC UA server URL (e.g., `opc.tcp://192.168.1.10:4840`)

Settings can be edited via the **Settings** screen within the app (admin only) or by manually editing the JSON store file.

6.5 CSV Integration (POS ↔ Conveyor)

The application integrates with your POS system entirely through the filesystem using CSV files.

Inbound — POS Order Import

Trigger: File watcher polls `posCsvDir` every 5 seconds. **Format:** SPOT POS export format.

When a new CSV is detected:

1. Each row is parsed as one garment line.
2. The system creates or updates the `customers`, `tickets`, and `garments` records.
3. On success, the source CSV file is **deleted**.
4. On parse failure, the file is left untouched (no partial writes).

Important fields expected in the CSV:

- Invoice number (full and display format)
- Customer ID, first name, last name, phone number
- Item barcode (`item_id`)
- Item description
- Dropoff date, pickup date
- Comments / special instructions

Outbound — Conveyor Activity CSV

Trigger: Written synchronously on each conveyor operation. **Destination:** `conveyorOutputDir`.

Four operation types are written as CSV records:

Operation	Trigger
`LoadItem`	Garment assigned to a slot
`UnloadItem`	Garment flagged for removal (ticket complete)
`Print`	Receipt print triggered
`SplitInvoice`	Invoice split operation performed

These files are intended for downstream systems (e.g., sorter controllers, reporting tools).

6.6 Hardware Integration

OPC UA Server

- **Library:** `open62541` 0.10
- **Protocol:** OPC UA (TCP)
- **Poll frequency:** 500 ms
- **Reconnect policy:** 3-second backoff on disconnect

Nodes read from OPC:

- System online/offline flag
- Hanger sensor state (garment detected at load point)
- Target slot number (current conveyor position)

Commands sent to OPC:

- Slot jog request (move conveyor to a specific slot)
- Slot run request

If the OPC server is unreachable, the application continues functioning in a **degraded mode** — garments can be scanned and slots assigned, but physical conveyor movement is not automated.

PLC / MODBUS

- **Library:** `tokio-modbus`
- **Protocol:** MODBUS TCP or RTU
- **Alternate protocol:** Mitsubishi MC Protocol via `melsec_mc`

Commands:

- Read current slot ID
- Write M5 command (trigger slot jog)

Thermal Printer (ESC/POS)

- **Library:** `escpos-rs` 0.4.3
- **Interface:** USB or serial port
- **Trigger:** Automatic on ticket completion; manual from Print Tickets screen

---

6.7 Slot State Machine

Each of the 100 slots follows this state machine:

```
┌──────────────────────────────────────────────────┐
│                     EMPTY                        │
│              (default, available)                │
└──────────────────────┬───────────────────────────┘
                       │ order scanned, slot allocated
                       ▼
┌──────────────────────────────────────────────────┐
│                   RESERVED                       │
│           (allocated, not yet physically loaded) │
└──────────────────────┬───────────────────────────┘
                       │ garment hung / barcode confirmed
                       ▼
┌──────────────────────────────────────────────────┐
│                   OCCUPIED                       │
│              (garment physically present)        │
└──────────────────────┬───────────────────────────┘
                       │ ticket complete / clear
                       ▼
┌──────────────────────────────────────────────────┐
│                     EMPTY                        │
└──────────────────────────────────────────────────┘

Any state ──► BLOCKED  (manually blocked by admin)
Any state ──► ERROR    (hardware/data inconsistency detected)
BLOCKED / ERROR ──► EMPTY  (admin clears)
```

Slot allocation uses a **round-robin** strategy starting from `last_used_slot + 1`, wrapping at slot 100 back to slot 1. Empty slots are preferred; reserved and occupied slots are skipped.

6.8 Session & Authentication Model

- Authentication is **PIN-based** (4 numeric digits).
- PINs are stored as text in the `users` table. Consider database-level access controls to protect this data.
- On login:
  - A `sessions` row is created with `login_at = NOW()`.
  - Any previous unclosed session for the same user is closed automatically.
- On logout:
  - The session `logout_at` is set to `NOW()`.
  - `garments_scanned` and `tickets_completed` are finalized.
- Sessions remain readable in the database indefinitely for audit purposes.

6.9 Database Migrations

Migrations are managed by **Diesel** and run automatically when the application successfully connects to the database for the first time (or after a software update that includes new migrations).

Migration files are embedded in the binary. No manual migration steps are required in normal operations.

If a migration fails:

1. The app logs the error and refuses to open.
2. Connect to PostgreSQL directly and inspect the `__diesel_schema_migrations` table.
3. Ensure the database user has `CREATE TABLE`, `ALTER TABLE`, and `INSERT` privileges.
4. If a migration is stuck in a partially-applied state, it may need to be manually rolled back. Contact the developer.

Migration history summary (14 migrations, Dec 2025 – Feb 2026):

- Initial schema (users, customers, tickets, garments)
- Slot management (slots table, app_state table)
- Session tracking
- Conveyor activity log
- `is_admin` field addition
- Split invoice support

6.10 Build & Deployment

The application is distributed as a self-contained installer.

Windows:

- MSI installer (via NSIS)
- All Rust and WebView dependencies are bundled

CI/CD:

- Builds are produced via **GitHub Actions** on push to `main`.
- Separate build jobs for Windows (x64).

To build locally:

```
# Prerequisites: Node.js 20+, Rust toolchain (stable), PostgreSQL dev libraries

# Install frontend dependencies
npm install

# Development (hot reload)
npm run tauri dev

# Production build
npm run tauri build
```

> *Ensure `libpq` (PostgreSQL client library) is installed on the build host, as Diesel requires it to link against.*

6.11 Log Files & Troubleshooting

Log locations:

Platform	Path
Windows	`%APPDATA%\com.whiteconveyors.conveyoros-oas\logs\`

Log levels: ERROR, WARN, INFO, DEBUG (controlled via Tauri log plugin configuration).

Common issues:

Symptom	Likely Cause	Resolution
Setup wizard loops repeatedly	Settings file corrupted or database unreachable	Delete the settings store file and re-run setup
"Garment Not Found" on valid barcode	POS CSV not yet imported	Check `posCsvDir` is correct and the CSV was received
OPC status always shows Offline	OPC server down, wrong URL, or firewall	Verify OPC endpoint URL in Settings; check firewall rules on port 4840
Slot state stuck in "Reserved"	Crash during scan before slot confirmed	Use the Slot view to manually clear affected slots
Tickets not printing	Printer offline or wrong port	Check printer connection; verify ESC/POS printer is configured
Migration error on startup	Insufficient database permissions	Grant `CREATE`, `ALTER`, `INSERT`, `SELECT` to the database user
CSV not being imported	Watcher path incorrect or file permissions	Verify `posCsvDir` is the exact directory; check read permissions

---

7. Workflows — End-to-End

7.1 Order Scanning Workflow

```
Operator logs in
       │
       ▼
Navigate to Scan screen
       │
       ▼
Scan garment barcode (scanner or keyboard)
       │
       ├─► Garment NOT found ──► Show error, wait for next scan
       │
       ├─► Garment already on conveyor
       │       └─► Re-send jog command to existing slot
       │           Show slot number to operator
       │
       └─► Garment found, not yet on conveyor
               │
               ├─► Is this the LAST garment on the ticket?
               │       │
               │       ├─► NO:
               │       │       Reserve next empty slot
               │       │       Write LoadItem to output CSV
               │       │       Show slot number, customer info
               │       │       Increment session garment count
               │       │
               │       └─► YES:
               │               Reserve slot for final garment
               │               Write LoadItem for final garment
               │               Write UnloadItem for ALL garments on ticket
               │               Mark ticket status = "Complete"
               │               Write Print command to output CSV
               │               Send print command to printer
               │               Increment tickets_completed in session
               │               Show "Ticket Complete" screen
               │
       ▼
Wait for next scan
```

7.2 CSV Order Import Workflow

```
POS system writes order CSV to posCsvDir
       │
       ▼ (file watcher polls every 5 seconds)
New file detected
       │
       ▼
Parse each row:
  - Extract customer data → Upsert into customers table
  - Extract ticket data → Create ticket if not exists
  - Extract garment data → Create garment records
       │
       ├─► Parse SUCCESS → Delete source CSV file
```

```
       │
       └─► Parse FAILURE → Leave CSV untouched, log error
                           Operator must fix and re-deliver the file
```

7.3 Conveyor Clearing Workflow

```
Admin/Operator initiates Clear
       │
       ▼
Confirmation dialog → Enter code "123"
       │
       ▼
For each slot in state "occupied" or "reserved":
  │
  ├─► Send jog command to PLC (move slot to exit)
  ├─► Operator physically removes garment
  ├─► Set garment.slot_number = -1
  ├─► Set slot.slot_state = "empty"
  └─► Reset ticket.garments_processed = 0
       │
       ▼
Reset app_state.num_items_on_conveyor = 0
       │
       ▼
Conveyor is now fully cleared and ready
```

---

8. Roles & Permissions Reference

Capability	Operator	Admin Operator
Login with PIN	Yes	Yes
Scan garments	Yes	Yes
View dashboard	Yes	Yes
Browse customer/order data	Yes	Yes
Print tickets	Yes	Yes
Clear conveyor	Yes	Yes
View slot map	Yes	Yes
Create operator accounts	No	Yes
View/edit settings (DB, OPC, paths)	No	Yes
Run database connection test	No	Yes
Access operator performance analytics	No	Yes

---

9. Glossary

Term	Definition
Barcode / Item ID	A unique identifier printed on each garment's tag, scanned by the operator.
Conveyor	The physical motorized conveyor system with numbered slots for hanging garments.
Garment	An individual item of clothing belonging to a customer order.
Invoice / Ticket	A customer order containing one or more garments. Used interchangeably.
MODBUS	An industrial serial communication protocol used to interface with PLCs.
OPC / OPC UA	OLE for Process Control — a standard for industrial data exchange. Provides real-time conveyor status.
PLC	Programmable Logic Controller — the embedded computer that controls the physical conveyor motor and sensors.
POS	Point of Sale — the system your business uses to check in customer orders (e.g., SPOT).
Session	A login period for one operator, from login to logout, with tracked productivity metrics.
Slot	One position on the conveyor where a garment can be hung. Numbered 1–100.
Slot State	The current status of a conveyor slot: empty, reserved, occupied, blocked, or error.
SPOT	A POS software commonly used in dry cleaning; the CSV import format is based on SPOT exports.
Tauri	The application framework used to build ConveyorOS OAS (Rust backend + web frontend).
Ticket Complete	The state reached when every garment on an order has been loaded onto the conveyor.
OAS	Order Assembly System — the full name of this application.